

ClinicFlow

Clinic appointment booking — from a developer laptop to a monitored, GitOps-managed EKS cluster

25
automated tests

8
pipeline jobs

69
AWS resources in Terraform

0
fixable HIGH/CRITICAL CVEs

Ankit Kumar · DevOps Heroes — Session 21 Final Capstone · github.com/ankit14-dev/clinicflow

What ClinicFlow does

The front-desk system of a small multi-specialty clinic: receptionists register patients, book them with a doctor, check them in on arrival and close the visit.



Safe booking

Overlap check per doctor — a double booking is rejected with HTTP 409; only future slots



Front-desk workflow

SCHEDULED → CHECKED_IN → COMPLETED, or CANCELLED / NO_SHOW — invalid jumps are refused



Live dashboard

Today's schedule, waiting room, upcoming visits and load per specialty, in clinic time (IST)



Observable

/health, /ready and /metrics, plus business counters such as bookings per specialty

Tech stack



Frontend

React 19 + Vite 8, responsive CSS, served by unprivileged nginx



Backend

FastAPI, SQLAlchemy 2, Pydantic 2, Alembic migrations



Data

PostgreSQL 17 — StatefulSet on EBS gp3 in the cluster



Containers

Multi-stage Docker builds, non-root, Docker Compose locally



CI/CD & security

GitHub Actions, Trivy, GHCR, kind, Dependabot, OIDC to AWS



Cloud & IaC

Terraform: VPC + EKS 1.36 in ap-south-1, IAM, EBS CSI



Kubernetes & GitOps

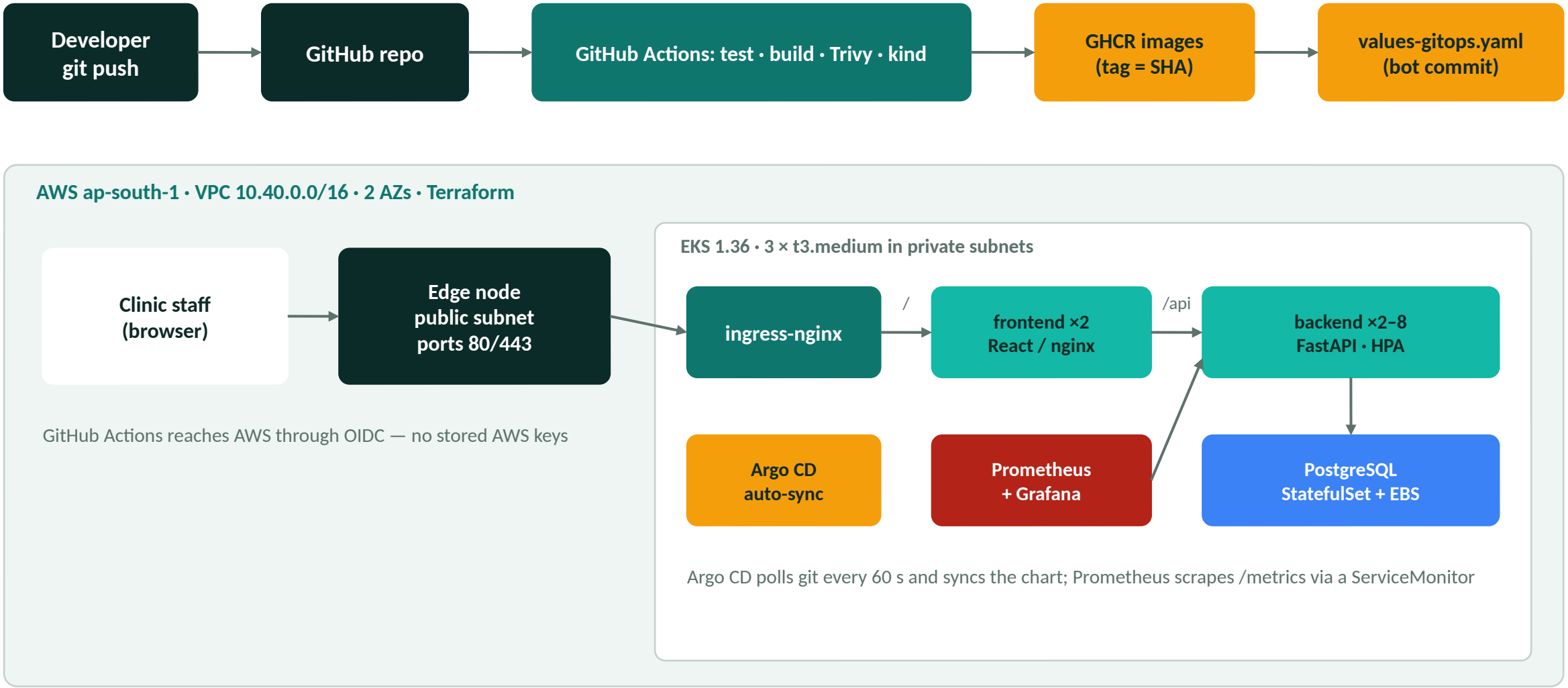
Helm chart, ingress-nginx, HPA, PDBs, Argo CD



Observability

kube-prometheus-stack: Prometheus, Grafana, alert rules

Architecture



Application and REST API

Endpoint	Purpose
`GET /health · /ready · /metrics`	liveness · DB readiness (503 if down) · Prometheus
`GET /api/stats`	dashboard KPIs in the clinic's timezone
`GET/POST /api/doctors, /api/patients`	list, search (?q=), register
`GET/PUT/DELETE /api/{resource}/{id}`	read, update, delete (409 if history exists)
`GET/POST /api/appointments`	filter by status / doctor / day; book (409 on overlap)
`PUT /api/appointments/{id}`	reschedule or change status — validated transitions



Rules the API enforces

- No overlapping bookings for a doctor
- Back-to-back slots are allowed
- Only future times; off-duty doctors can't be booked
- Status state machine, terminal states locked
- Schema via Alembic: 3 tables, check constraints, indexes

Tests are the first quality gate

25	99%	≥90%
pytest cases	line coverage	required by CI

- CI runs pytest against a throwaway PostgreSQL service container — never a real database
- Also in CI: ruff lint/format and alembic upgrade → check → downgrade
- Covers CRUD, 422 validation, 404s, double booking, rescheduling conflicts, the status machine, filters, stats and /metrics

```
Bad code
   ↓
pytest fails
   ↓
pipeline stops
   ↓
no image is built or
pushed
```

Containers: small, non-root, scanned

`clinicflow-backend` — 313 MB

- python:3.14-slim, 2 stages: venv built, then copied into a clean image
- Runs as uid 10001; pip removed from the runtime image
- Starts with alembic upgrade head, then exec uvicorn

`clinicflow-frontend` — 90 MB

- node:24-alpine builds dist/, nginx-unprivileged:1.30 serves it
- Runs as uid 101 on port 8080; proxies /api to the backend
- Security headers, 1-year cache for hashed assets

In Kubernetes every pod also runs with a read-only root filesystem, no Linux capabilities and seccomp RuntimeDefault — the namespace enforces Pod Security "restricted". Locally: docker compose up --build → http://localhost:3000

CI/CD pipeline — every push to main

1 Test

pytest + PostgreSQL, ruff, migrations

2 Build

frontend build, npm audit, helm lint, terraform validate

3 Image

docker build backend + frontend

4 Scan

Trivy gate: fail on fixable HIGH/CRITICAL

5 Push

GHCR, tag = full commit SHA

6 Smoke

kind cluster: helm install + helm test

7 Promote

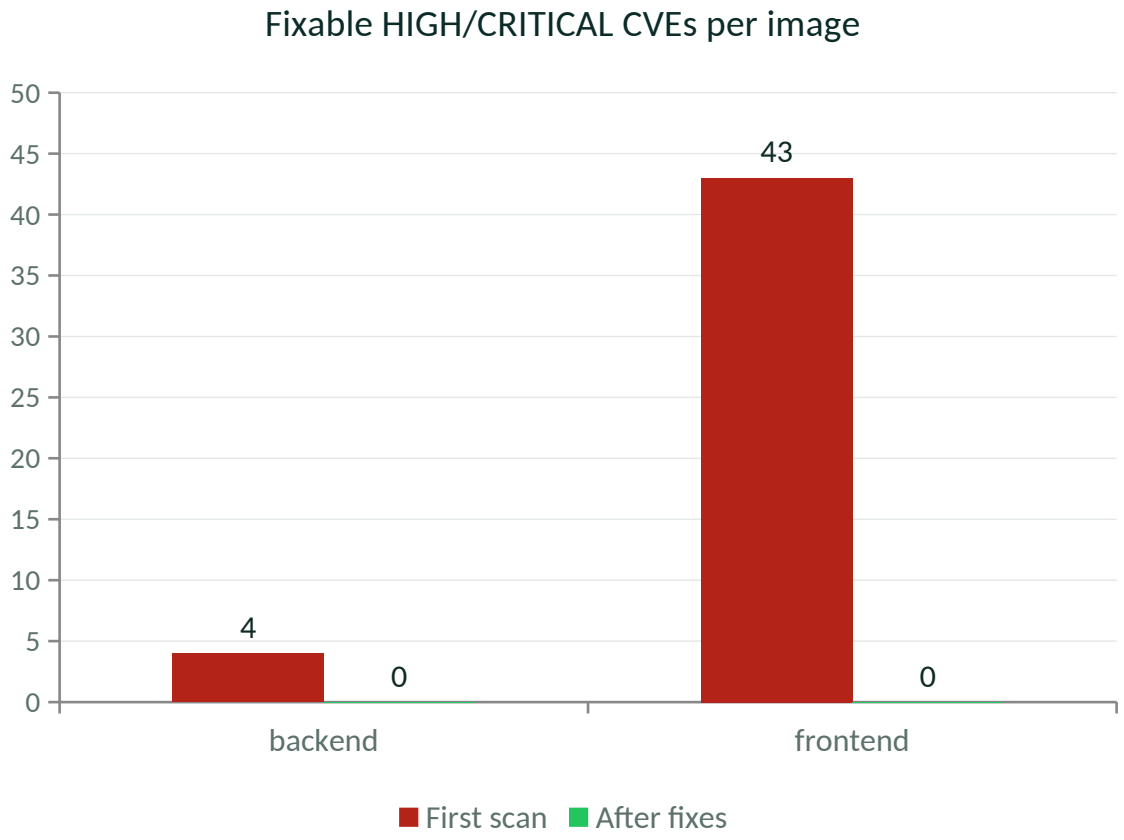
bot commits tag to values-gitops.yaml

8 Verify

OIDC → wait for Argo CD rollout, smoke test

All third-party actions pinned to commit SHAs · Dependabot keeps them current · PRs run everything except push/deploy

DevSecOps: Trivy blocks vulnerable images



Backend: 4 HIGH

urllib3 CVE-2026-97687 / -97689 and msgpack GHSA-6v7p-g79w-8964 — inside pip's own vendored copies. The app never needs pip at runtime, so it is removed from the image.

Frontend: 43 HIGH

Old Alpine packages (curl, c-ares, …) in the nginx base image. Moving to the nginx 1.30 stable base removed all of them.

Layers: npm audit + Trivy (dependencies) · Trivy gate + SARIF to GitHub Security (containers) · secret scanning · SHA-pinned actions · OIDC instead of AWS keys · restricted pods

Infrastructure as Code — Terraform on AWS

69
resources

2
availability zones

3 + 1
nodes (app + edge)

≈ $0.32
per hour

Network

VPC 10.40.0.0/16, public + private subnets in 2 AZs, Internet Gateway, one NAT gateway

EKS

Kubernetes 1.36, managed node group 3 × t3.medium in private subnets; add-ons: VPC CNI, CoreDNS, kube-proxy, Pod Identity, EBS CSI, metrics-server

IAM

EBS CSI role via Pod Identity; GitHub OIDC provider + deploy role trusted only for this repo's production environment

terraform init → plan → apply (~15 min) · scripts/eks-down.sh removes Kubernetes-created volumes, then terraform destroy

Kubernetes, Helm and GitOps with Argo CD

Helm chart: helm/clinicflow

- Deployments: backend 2–8 pods (HPA), frontend 2 pods, zero-downtime rolling updates
- Startup, readiness (/ready checks DB) and liveness probes
- PostgreSQL StatefulSet + 5 Gi gp3 PersistentVolume
- Ingress: / → frontend, /api → backend
- PodDisruptionBudgets, ServiceMonitor, alert rules, Grafana dashboard

GitOps flow

1. CI pushes scanned image ghcr.io/…:SHA
2. CI commits the tag to values-gitops.yaml
3. Argo CD sees the new commit (polls every 60 s)
4. Argo CD renders the chart and syncs EKS
5. Self-heal reverts manual drift · rollback = git revert

Observability and autoscaling

Prometheus + Grafana

- ServiceMonitor scrapes /metrics on every backend pod — Targets show UP
- Dashboard provisioned from the chart: requests/s, p50/p95/p99 latency, 5xx ratio, bookings per specialty, double bookings rejected, HPA replicas, CPU and memory per pod
- Alerts: backend down, error ratio > 5 %, p95 > 500 ms, HPA at max, restart loops

HPA under load on EKS

388 %

CPU of request

2 → 4

backend pods

Target 60 % of a 100m CPU request, 2–8 replicas. An in-cluster load-test Job drives the load; metrics-server feeds the HPA, and pods spread across the 3 nodes.

Real problems I solved on AWS

1. **Account cannot create load balancers**
   AWS returned OperationNotPermitted for every ELB. Added a small tainted edge node in a public subnet running ingress-nginx on ports 80/443; app nodes stay private.

2. **Edge node: port 80 'in use'**
   The EKS Pod Identity agent declares hostNetwork port 80. Kept it off the edge node with an add-on affinity rule in Terraform.

3. **GitHub OIDC refused**
   The repo uses GitHub's immutable subject format (owner and repo IDs). The IAM trust policy now accepts both formats.

4. **Migration race on first deploy**
   Two replicas created the schema at once; one crashed. A PostgreSQL advisory lock makes migrations take turns — tested with 4 in parallel.

5. **Database restarted on every deploy**
   The StatefulSet carried the image SHA as a label. Version-free labels: app deploys no longer touch PostgreSQL.



Key learnings

...and what I'd add for production

- Quality and security gates must run before an image can reach the registry
- GitOps makes git the audit trail: every deploy is a commit, every rollback a revert
- Readiness and liveness are different questions — and both matter in rollouts
- Cloud accounts have limits IAM can't fix — design around them in code

Next: Amazon RDS Multi-AZ · TLS with a domain and cert-manager · one NAT per AZ · Argo CD SSO · remote Terraform state